

FastAPI Interview Q&A

Complete Guide: Questions 1–35 with Detailed Answers

Beginner Questions (1–10)

Q1. What is FastAPI and why would you choose it over Flask or Django?

FastAPI is a modern Python web framework for building APIs quickly and efficiently. Created by Sebastián Ramírez, it is built on top of **Starlette** (web functionality) and **Pydantic** (data validation), and uses Python type hints to automatically validate requests and generate API documentation.

FastAPI vs Flask

Feature	FastAPI	Flask
Async Support	Yes	Limited
Validation	Built-in	Manual
Documentation	Auto Generated	Manual
Performance	Very High	Moderate
Type Safety	Yes	No

FastAPI vs Django

Django is a full framework including Authentication, ORM, Admin Panel, and Sessions. FastAPI focuses mainly on APIs. Choose Django for complete web applications; choose FastAPI for microservices or APIs.

Interview Answer: FastAPI is a high-performance Python API framework that provides automatic validation, automatic API documentation, type safety, and async support, making it ideal for modern microservices and scalable backend applications.

Q2. What is ASGI and how is it different from WSGI?

WSGI (Web Server Gateway Interface)

Used by Flask and traditional Django. Handles one request at a time synchronously. Threads block on I/O operations like external API calls.

ASGI (Asynchronous Server Gateway Interface)

Used by FastAPI and Starlette. Supports async/await, WebSockets, and background tasks. While one request waits for I/O, other requests continue processing.

Feature	WSGI	ASGI
Sync	Yes	Yes
Async	No	Yes
WebSocket	No	Yes

Performance	Moderate	High
-------------	----------	------

Interview Answer: WSGI supports only synchronous request handling, whereas ASGI supports both synchronous and asynchronous communication, enabling high concurrency, WebSockets, and better scalability.

Q3. How does FastAPI achieve high performance?

- **Starlette:** Extremely lightweight web toolkit underneath FastAPI.
- **ASGI:** Allows async/await instead of blocking threads.
- **Event Loop:** Multiple requests processed concurrently via Python's event loop.
- **Pydantic Validation:** Optimized automatic validation, parsing, and conversion with minimal overhead.

Interview Answer: FastAPI achieves high performance through ASGI, Starlette, asynchronous execution, optimized request handling, and efficient Pydantic-based validation.

Q4. Explain Path Parameters and Query Parameters

Path Parameters

Part of the URL path — used to identify a specific resource.

```
@app.get("/users/{id}")
def get_user(id: int):
    return id

# Request: /users/101
```

Query Parameters

Appear after '?' — used for filtering, searching, sorting, and pagination.

```
@app.get("/users")
def get_users(page: int):
    return page

# Request: /users?page=2
```

Interview Answer: Path parameters identify a resource, while query parameters modify or filter results.

Q5. How does FastAPI validate incoming request data?

FastAPI uses Pydantic models for automatic validation. Invalid data is rejected before reaching business logic.

```
class User(BaseModel):
    name: str
    age: int

@app.post("/users")
def create_user(user: User):
    return user
```

Sending **{"age": "abc"}** returns an automatic validation error with descriptive messages.

Interview Answer: FastAPI validates incoming requests using Pydantic models and Python type hints, automatically rejecting invalid data before it reaches business logic.

Q6. What is Pydantic and why is it important?

Pydantic is a Python library for data validation, serialization, and parsing. It automatically converts compatible types (e.g., string '22' to integer 22) and raises detailed errors for invalid data.

Without Pydantic, you write manual validation with `isinstance()` checks everywhere. With Pydantic, FastAPI handles it automatically.

Interview Answer: Pydantic provides automatic validation, serialization, type conversion, and schema generation, making FastAPI development faster and safer.

Q7. What is Uvicorn?

Uvicorn is a lightweight ASGI server — think of it as the engine that powers FastAPI. Without Uvicorn, FastAPI cannot receive requests.

```
uvicorn main:app --reload
```

Key features: ASGI compatible, high performance, async support, WebSockets.

Interview Answer: Uvicorn is a lightweight ASGI server responsible for running FastAPI applications and handling incoming client requests.

Q8. How does FastAPI generate Swagger and ReDoc automatically?

FastAPI reads routes, type hints, and Pydantic models to create an OpenAPI schema. It then exposes:

- /docs — Swagger UI for interactive testing
- /redoc — ReDoc UI for clean documentation

No manual documentation needed — frontend developers can test APIs directly.

Interview Answer: FastAPI automatically generates OpenAPI specifications and uses them to create Swagger UI and ReDoc documentation.

Q9. Difference between POST, PUT and PATCH

Method	Purpose	Example
POST	Create new resource	POST /users
PUT	Replace entire resource (send complete object)	PUT /users/1
PATCH	Update specific fields only	PATCH /users/1

Interview Answer: POST creates resources, PUT replaces entire resources, and PATCH partially updates existing resources.

Q10. How do you return custom HTTP status codes and errors?

```

from fastapi import HTTPException

raise HTTPException(
    status_code=404,
    detail="User not found"
)

```

Code	Meaning
200	Success
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

Interview Answer: FastAPI uses HTTPException and custom responses to return meaningful status codes and error messages, improving API usability and debugging.

Intermediate Questions (11–25)

Q11. Explain FastAPI Dependency Injection. Why is it useful?

Dependency Injection (DI) is a design pattern where a function receives its dependencies from an external source instead of creating them itself. Without DI, you repeat `db = Database()` across 100 APIs. With DI, FastAPI automatically provides the dependency.

```

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@app.get("/users")
def get_users(db: Session = Depends(get_db)):
    return db.query(User).all()

```

Advantages: Code Reusability, Easy Testing (mock dependencies), Separation of Concerns, Better Maintainability.

Interview Answer: Dependency Injection in FastAPI allows reusable components such as database sessions, authentication logic, and configurations to be automatically provided to route handlers using `Depends()`, improving code reusability, testing, and maintainability.

Q12. Difference Between `Depends()` and Direct Function Calls

Direct function calls are normal Python — no lifecycle management, no caching, no validation. Using Depends() integrates with FastAPI's dependency system:

- Creates and injects the dependency
- Cleans up after the request
- Handles caching
- Performs validation

Interview Answer: Depends() integrates with FastAPI's dependency system and provides automatic lifecycle management, validation, and reusability, whereas direct function calls do not.

Q13. When should you use async def instead of def?

Operation Type	Use	Examples
I/O Bound	async def	DB calls, External APIs, Redis, File Uploads, S3
CPU Bound	def	Image Processing, Data Compression, ML Computation

Using async def for CPU-bound work provides no benefit and can actually block the event loop.

Interview Answer: Use async def for I/O-bound operations such as database queries, API calls, and file operations. Use def for CPU-intensive tasks because asynchronous execution does not improve CPU-bound performance.

Q14. Explain the FastAPI Request Lifecycle

- **Step 1:** Client sends request
- **Step 2:** Middleware executes (Logging, Authentication, Metrics)
- **Step 3:** Dependencies resolve (DB Session, User Context)
- **Step 4:** Pydantic validates request data
- **Step 5:** Route handler executes business logic
- **Step 6:** Response is generated and returned
- **Step 7:** Cleanup — DB connections and resources closed

Interview Answer: Request Lifecycle consists of middleware execution, dependency resolution, request validation, route execution, response generation, and resource cleanup.

Q15. How Would You Implement JWT Authentication?

JWT (JSON Web Token) enables stateless authentication. Flow:

- User submits credentials → server validates → generates signed token
- Client stores token and sends in Authorization header: **Bearer <token>**
- FastAPI decodes and validates token on each request

```
# Login
token = jwt.encode(payload, SECRET_KEY)
return {"access_token": token}
```

```
# Protected route
jwt.decode(token, SECRET_KEY)
```

Interview Answer: JWT Authentication works by generating signed tokens after login. Clients send the token in the Authorization header, and FastAPI validates it before granting access to protected routes.

Q16. Explain Access Tokens and Refresh Tokens

Token Type	Lifespan	Purpose
Access Token	~15 minutes	API authorization
Refresh Token	~7 days	Generate new access tokens

Without refresh tokens, users must log in every 15 minutes — poor UX. Refresh tokens allow seamless re-authentication. If an access token leaks, damage is limited to its short lifespan.

Interview Answer: Access tokens are short-lived tokens used for authorization, while refresh tokens are long-lived tokens used to obtain new access tokens without forcing users to log in again.

Q17. How Do You Implement RBAC?

RBAC (Role-Based Access Control) restricts access based on user roles (Admin, Manager, User).

```
def require_admin(user = Depends(get_current_user)):
    if user.role != "admin":
        raise HTTPException(403)

@app.delete("/users")
def delete_user(user = Depends(require_admin)):
    pass
```

Interview Answer: RBAC restricts access based on user roles. FastAPI typically implements RBAC using dependencies that verify user permissions before allowing access to protected resources.

Q18. What is CORS and Why Does it Exist?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism. When frontend (localhost:3000) and backend (localhost:8000) are on different origins, browsers block requests by default.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"]
)
```

Interview Answer: CORS is a browser security mechanism that controls cross-origin requests. It prevents unauthorized websites from accessing resources from another domain.

Q19. How Do You Organize a Large FastAPI Project Using APIRouter?

```
app/  
├── routers/  
│   ├── auth.py  
│   ├── users.py  
│   └── products.py  
├── services/  
├── models/  
├── schemas/  
├── database/  
└── main.py
```

```
# In users.py  
router = APIRouter()
```

```
# In main.py  
app.include_router(users_router, prefix="/users")
```

Interview Answer: APIRouter helps modularize routes into separate files and domains, improving maintainability and scalability for large FastAPI applications.

Q20. How Do You Manage Configuration and Environment Variables?

Never hardcode secrets in source code. Use environment variables and Pydantic Settings:

```
# .env  
DATABASE_URL=postgresql://...  
SECRET_KEY=abc  
  
# settings.py  
class Settings(BaseSettings):  
    SECRET_KEY: str  
    DATABASE_URL: str
```

Interview Answer: FastAPI applications typically manage configuration using environment variables and Pydantic Settings, ensuring security and flexibility across environments.

Q21. Explain Middleware and Provide Real-World Use Cases

Middleware is code that executes before and after every request — like a checkpoint.

```
@app.middleware("http")  
async def log_requests(request: Request, call_next):  
    start = time.time()  
    response = await call_next(request)  
    print(time.time() - start)  
    return response
```

- Logging — track all requests
- Authentication — check API keys before routes
- Rate Limiting — prevent abuse
- Monitoring — track response times

- Correlation IDs — useful in microservices

Interview Answer: Middleware executes before and after request processing and is commonly used for logging, authentication, monitoring, rate limiting, and request tracing.

Q22. How Do You Upload and Process Files in FastAPI?

```
from fastapi import UploadFile

@app.post("/upload")
async def upload_file(file: UploadFile):
    content = await file.read()
    with open(file.filename, "wb") as f:
        f.write(content)
    return {"filename": file.filename}
```

Real-world examples: Resume uploads (PDF), profile image uploads (PNG/JPG), Excel file processing. Always validate file size, type, and perform virus scanning in production.

Interview Answer: FastAPI uses UploadFile for efficient file uploads. Files can be streamed, validated, stored, and processed asynchronously.

Q23. How Do You Handle Validation Errors and Custom Exceptions?

FastAPI returns automatic validation errors for Pydantic failures. For custom errors:

```
raise HTTPException(status_code=404, detail="User not found")

# Global handler
@app.exception_handler(UserNotFoundException)
async def handler(request, exc):
    return JSONResponse(status_code=404,
                        content={"message": "User Missing"})
```

Interview Answer: FastAPI provides automatic validation error handling and allows custom exception handlers for centralized and consistent error management.

Q24. What are Pydantic Validators and When Would You Use Them?

When type validation isn't enough — e.g., an empty string is a valid str but shouldn't be accepted.

```
from pydantic import field_validator

class User(BaseModel):
    name: str

    @field_validator("name")
    def validate_name(cls, value):
        if len(value) < 3:
            raise ValueError("Too Short")
        return value
```


Use cases: password strength rules, age verification (18+), domain-specific email validation.

Interview Answer: Pydantic validators provide custom validation logic beyond type checking and are useful for enforcing business rules and data integrity.

Q25. How Would You Implement API Versioning?

When you change an API response structure, existing clients break. Versioning allows multiple versions to coexist.

```
app.include_router(v1_router, prefix="/api/v1")
app.include_router(v2_router, prefix="/api/v2")
```

```
# Client uses: /api/v1/users or /api/v2/users
```

Interview Answer: API versioning allows multiple API versions to coexist, ensuring backward compatibility while introducing new features.

Advanced Questions (26–35)

Q26. Explain Synchronous vs Asynchronous Database Operations

Approach	Behavior	Scalability
Synchronous	Thread blocked while waiting for DB	Requires many threads
Asynchronous	Event loop continues while waiting	High concurrency, few threads

Interview Answer: Synchronous operations block execution while waiting for results, whereas asynchronous operations allow other requests to be processed concurrently.

Q27. How Would You Integrate SQLAlchemy Async With FastAPI?

```
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession

engine = create_async_engine(DATABASE_URL)

async def get_db():
    async with SessionLocal() as db:
        yield db

@app.get("/users")
async def users(db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(User))
    return result.scalars().all()
```

Interview Answer: SQLAlchemy Async uses AsyncEngine and AsyncSession with async drivers such as asyncpg, enabling non-blocking database operations.

Q28. Explain Connection Pooling and Why It Matters

Creating a new DB connection for every request is expensive and slow. Connection pooling maintains a pool of reusable connections.

Without pooling: each request creates a new connection. With pooling: existing connections are reused from a pool. PostgreSQL often limits to ~100 connections — pooling prevents exhaustion.

Interview Answer: Connection pooling reuses database connections, reducing overhead and improving application performance and scalability.

Q29. What are Background Tasks? When Would You Use Celery Instead?

FastAPI BackgroundTasks — lightweight, runs in the same server process:

```
@app.post("/send-email")
def send(background_tasks: BackgroundTasks):
    background_tasks.add_task(send_email)
    return {"status": "queued"}
```

Celery — distributed task queue with Redis/RabbitMQ broker. Use for: image/video processing, reports, AI jobs, heavy workloads. BackgroundTasks are lost if the server crashes; Celery tasks survive.

Interview Answer: BackgroundTasks are suitable for lightweight asynchronous work, while Celery is preferred for distributed, reliable, and long-running tasks.

Q30. How Would You Implement Redis Caching?

Cache frequently accessed data in Redis (in-memory) to avoid hitting the database every time.

- GET /products takes 3 seconds from DB → cache result in Redis → next request returns in 50ms
- Use cases: Product lists, User profiles, Analytics, Search results
- Set TTL (time-to-live) to avoid stale data

Interview Answer: Redis caching stores frequently accessed data in memory, significantly reducing database load and improving response times.

Q31. Explain WebSockets and Build a Real-Time Chat Architecture

HTTP is request-response and closes. WebSockets maintain a persistent two-way connection.

```
@app.websocket("/chat/{room_id}")
async def chat(websocket: WebSocket, room_id: str):
    await websocket.accept()
    while True:
        data = await websocket.receive_text()
        await websocket.send_text(f"Echo: {data}")
```

Real-time chat architecture: FastAPI handles WebSocket connections → Redis Pub/Sub distributes messages → PostgreSQL stores chat history.

Interview Answer: WebSockets enable persistent two-way communication between client and server, making them ideal for chat systems, live notifications, and real-time dashboards.

Q32. How Do You Implement Rate Limiting in Production?

Without rate limiting, a single attacker can send 10,000 requests/sec and crash the server.

```
from slowapi import Limiter
limiter = Limiter(key_func=get_remote_address)

@app.get("/api/data")
@limiter.limit("100/minute")
async def data(request: Request):
    return {"data": "ok"}
```

Tools: SlowAPI library, Redis counters, or API Gateway (AWS API Gateway, Nginx). Limit per IP, per user, or per API key.

Interview Answer: Rate limiting restricts request frequency per client, preventing abuse and protecting application resources.

Q33. How Would You Write Integration Tests for FastAPI?

```
from fastapi.testclient import TestClient

client = TestClient(app)

def test_create_user():
    response = client.post("/users",
        json={"name": "Pawan", "age": 22})
    assert response.status_code == 201

def test_user_not_found():
    response = client.get("/users/9999")
    assert response.status_code == 404
```

Test: Authentication flows, Database interactions, API endpoints, Error responses and edge cases.

Interview Answer: Integration tests validate the interaction between APIs, dependencies, and databases, ensuring the system works as expected end-to-end.

Q34. Explain Lifespan Events and Startup/Shutdown Management

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup
    await connect_db()
    await init_redis()
    load_ml_model()
    yield
    # Shutdown
    await disconnect_db()
    await close_redis()
```

```
app = FastAPI(lifespan=lifespan)
```

Use cases: Database connections, Redis initialization, ML model loading, cleanup tasks.

Interview Answer: Lifespan events manage application startup and shutdown activities such as database initialization, cache setup, and resource cleanup.

Q35. Describe a Production-Grade FastAPI Deployment Architecture

A complete production setup:

Component	Role
Cloudflare	CDN + DDoS protection
Load Balancer	Distribute traffic across instances
Nginx	Reverse proxy + static files
FastAPI (x3)	Application layer (horizontally scaled)
PostgreSQL	Persistent data storage
Redis	Caching + message queuing
Celery	Background jobs
Prometheus + Grafana	Metrics and monitoring

Interview Answer: A production-grade FastAPI architecture includes load balancers, Nginx, multiple FastAPI instances, PostgreSQL, Redis, background workers, monitoring, logging, and container orchestration for scalability and reliability.

System Design Questions (2–5 Years Experience)

URL Shortener

- Generate unique short codes
- Store mapping in PostgreSQL
- Cache popular URLs in Redis
- Use FastAPI for redirect APIs

Authentication Service

- JWT + Refresh Tokens
- RBAC (Role-Based Access Control)
- Password hashing using bcrypt
- Redis token blacklist for logout

Notification Service

- FastAPI receives requests

- Celery processes jobs asynchronously
- Redis/RabbitMQ as message broker
- Separate Email/SMS worker processes

Scale FastAPI to 1M Requests/Day

- Horizontal scaling (multiple instances)
- Redis caching for hot data
- Async DB operations with SQLAlchemy
- Load balancers + CDN
- Database indexing and query optimization

Microservice Architecture

- User Service, Auth Service, Payment Service, Notification Service
- API Gateway as single entry point
- Message Queue (Kafka/RabbitMQ) for async communication
- Independent database per service
- Service discovery and health checks

These questions cover Junior to Senior FastAPI interviews. Companies using FastAPI include Netflix, Uber, Airbnb, and many startups.